BTS SIO - SLAM | ATELIER DE PROFESSIONNALISATION

RAPPORT DE MISSION : MINI-PROJET HÉRITAGE (PHP & C#)

Informations Générales

Groupe : Mohammed El Amiry, Lohan Dealmeida

Dépôt GitHub : https://github.com/Hxplan/MINIPROJ-heritage.git

Sommaire du Rapport

1) PARTIE PHP
- 1.1 Présentation du Domaine Fonctionnel
- 1.2 Diagramme de Classes UML
- 1.3 Base de Données MySQL (Script SQL & Captures phpMyAdmin)
- 1.4 Code Source des Classes Métiers PHP (Arme.class.php & Epee.class.php)
- 1.5 Code et Captures d'Écran des Tests PHP (test.php)

2) PARTIE C#
- 2.1 Présentation du Domaine Fonctionnel
- 2.2 Diagramme de Classes UML C#
- 2.3 Base de Données MySQL (même base que le PHP)
- 2.4 Code Source des Classes Métiers C# (Arme.cs & Epee.cs)
- 2.5 Code et Captures d'écran des Tests C# (Form1.cs)

1) PARTIE PHP

1.1 Présentation du Domaine Fonctionnel

Dans le cadre d'un jeu vidéo, on a un inventaire composé d'armes et d'épées avec un calcul dynamique de leur puissance.

• Classe mère (Arme) : Représente une arme de base définie par un id, un nom, des dégâts de base et un poids. Il y a un contrôle des valeurs positives, une méthode de calcul de puissance (dégâts de base divisés par le poids) et une méthode d'affichage __toString().

• Classe fille (Epee) : Hérite de la classe Arme en lui ajoutant une longueur de lame et une valeur de tranchant. Elle hérite des attributs de la classe mère via son constructeur parent et redéfinit la méthode calculerPuissance() en ajoutant la valeur de son tranchant au calcul parent, ainsi que la méthode __toString().

1.2 Diagramme de Classes (UML)

Ce diagramme de classes illustre la relation d'héritage entre la classe mère Arme et la classe fille Epee, avec les attributs privés (-) et les méthodes publiques (+) :

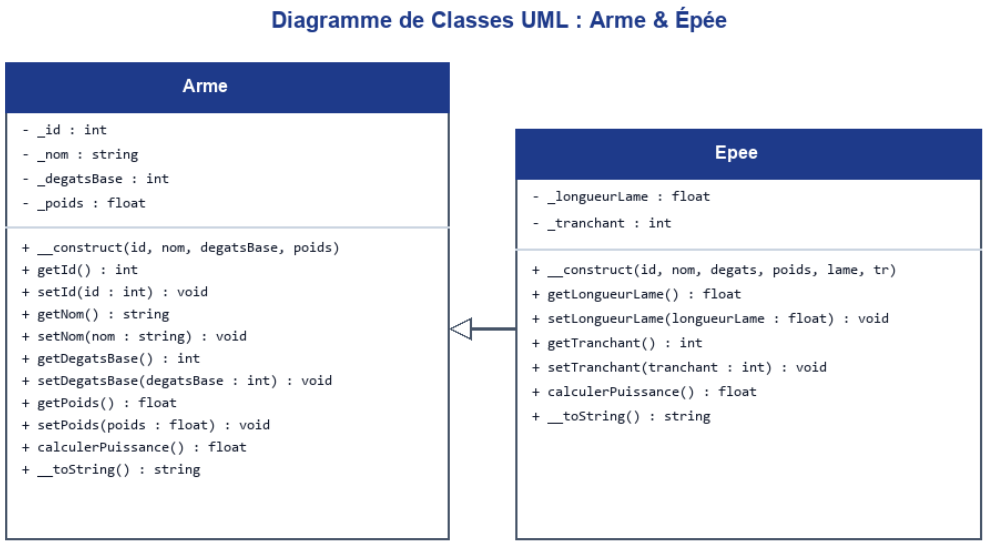


Figure 1 : Diagramme de classes UML (Arme et Epee)

1.3 Base de Données MySQL

Création des tables arme et epee dans la base de données miniprojet_heritage :

Fichier : SQL/script_heritage_armes.sql

```
-- 1. Création et de la base de données
CREATE DATABASE IF NOT EXISTS miniprojet_heritage DEFAULT CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE miniprojet_heritage;

-- Suppression des tables si elles existent déjà
DROP TABLE IF EXISTS epee;
DROP TABLE IF EXISTS arme;

-- =============================================================================
-- 2. Création de la table mère : arme
-- =============================================================================
CREATE TABLE arme (
    id INT AUTO_INCREMENT NOT NULL,
    nom VARCHAR(50) NOT NULL,
    degatsBase INT NOT NULL,
    poids DOUBLE NOT NULL,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- =============================================================================
-- 3. Création de la table fille : epee (hérite de arme)
-- =============================================================================
CREATE TABLE epee (
    id INT NOT NULL,
    longueurLame DOUBLE NOT NULL,
    tranchant INT NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_epee_arme FOREIGN KEY (id) REFERENCES arme(id) ON DELETE CASCADE ON UPDATE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Captures d'écran de la structure et des données des tables sous MySQL / phpMyAdmin :

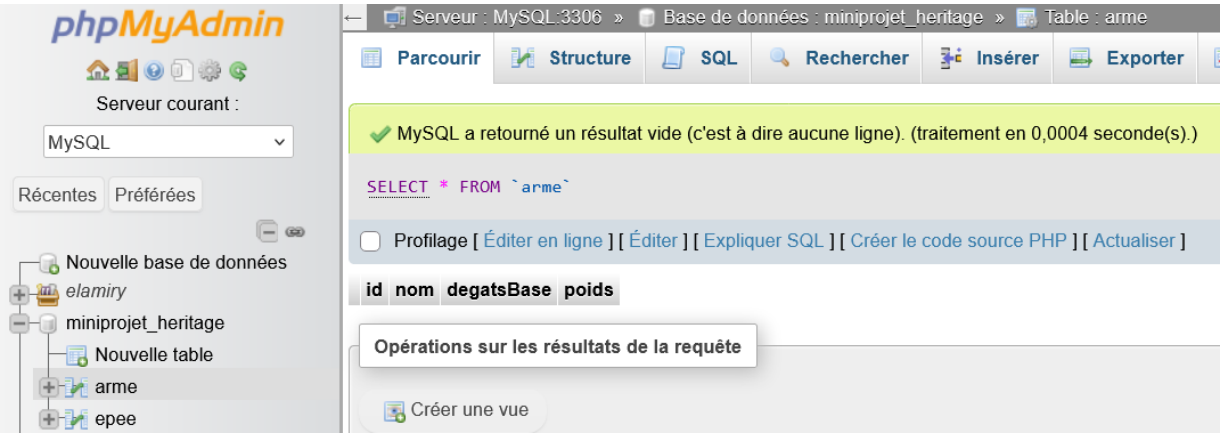


Figure 2 : Table arme sous phpMyAdmin

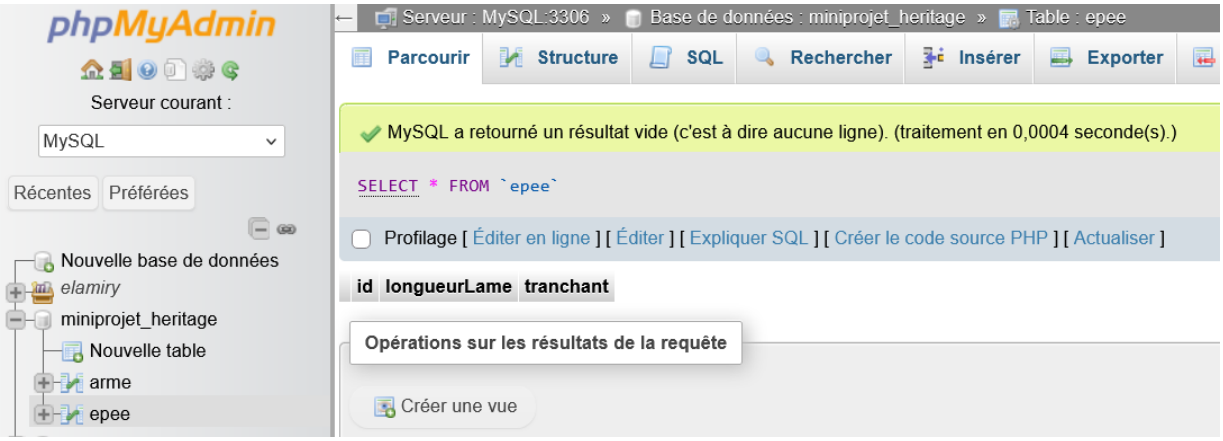


Figure 3 : Table epee sous phpMyAdmin

1.4 Code Source des Classes Métiers PHP

Classes développées sous VSCode respectant l'organisation en régions, les accesseurs, les exceptions et les méthodes redéfinies :

Fichier : PHP/Arme.class.php

```php
<?php

class Arme
{
    #region Champs

    private int $_id;
    private string $_nom;
    private int $_degatsBase;
    private float $_poids;

    #endregion

    #region Propriétés

    /**     get     **/
    public function __get(mixed $propriete)
    {
        switch ($propriete) {
            case "id":
                return $this->_id;
            case "nom":
                return $this->_nom;
            case "degatsBase":
```

```
            return $this->_degatsBase;
        case "poids":
            return $this->_poids;
        default:
            throw new Exception("propriété '$propriete' inconnue");
    }
}

/**     set     **/
public function __set(mixed $propriete, mixed $value)
{
    switch ($propriete) {
        case "id":
            $this->setId($value);
            break;
        case "nom":
            $this->setNom($value);
            break;
        case "degatsBase":
            $this->setDegatsBase($value);
            break;
        case "poids":
            $this->setPoids($value);
            break;
        default:
            throw new Exception("propriété '$propriete' inconnue");
    }
}

#endregion

#region Constructeur

public function __construct(int $id, string $nom, int $degatsBase, float $poids)
{
    $this->setId($id);
    $this->setNom($nom);
    $this->setDegatsBase($degatsBase);
    $this->setPoids($poids);
}

public function __destruct()
{
}

#endregion

#region Assesseurs (getteur/setteur)

/**     get/set de id     **/
public function getId(): int
{
    return $this->_id;
}

public function setId(int $id): void
{
    $this->_id = $id;
}

/**     get/set de nom     **/
public function getNom(): string
{
    return $this->_nom;
}
```

```
    public function setNom(string $nom): void
    {
        $this->_nom = $nom;
    }

    /**     get/set de degatsBase     **/
    public function getDegatsBase(): int
    {
        return $this->_degatsBase;
    }

    public function setDegatsBase(int $degatsBase): void
    {
        if ($degatsBase <= 0) {
            throw new Exception("Les dégâts doivent être positifs.");
        }
        $this->_degatsBase = $degatsBase;
    }

    /**     get/set de poids     **/
    public function getPoids(): float
    {
        return $this->_poids;
    }

    public function setPoids(float $poids): void
    {
        if ($poids <= 0) {
            throw new Exception("Le poids doit être positif.");
        }
        $this->_poids = $poids;
    }

    #endregion

    #region Méthodes

    public function calculerPuissance(): float
    {
        return $this->_degatsBase / $this->_poids;
    }

    #endregion

    #region Méthodes redéfinies

    public function __toString(): string
    {
        return "[" . $this->_id . "] " . $this->_nom . " - Dégâts: " . $this->_degatsBase . ",
Poids: " . $this->_poids . " kg";
    }

    public function toString(): string
    {
        return $this->__toString();
    }

    #endregion
}
?>
```

Fichier : PHP/Epee.class.php

```php
<?php

require_once "Arme.class.php";

class Epee extends Arme
{
    #region Champs

    private float $_longueurLame;
    private int $_tranchant;

    #endregion

    #region Propriétés

    /**     get     **/
    public function __get(mixed $propriete)
    {
        switch ($propriete) {
            case "longueurLame":
                return $this->_longueurLame;
            case "tranchant":
                return $this->_tranchant;
            default:
                return parent::__get($propriete);
        }
    }

    /**     set     **/
    public function __set(mixed $propriete, mixed $value)
    {
        switch ($propriete) {
            case "longueurLame":
                $this->setLongueurLame($value);
                break;
            case "tranchant":
                $this->setTranchant($value);
                break;
            default:
                parent::__set($propriete, $value);
        }
    }

    #endregion

    #region Constructeur

    public function __construct(int $id, string $nom, int $degatsBase, float $poids, float
$longueurLame, int $tranchant)
    {
        parent::__construct($id, $nom, $degatsBase, $poids);
        $this->setLongueurLame($longueurLame);
        $this->setTranchant($tranchant);
    }

    public function __destruct()
    {
    }

    #endregion

    #region Assesseurs (getteur/setteur)
```

```
    /**     get/set de longueurLame     **/
    public function getLongueurLame(): float
    {
        return $this->_longueurLame;
    }

    public function setLongueurLame(float $longueurLame): void
    {
        if ($longueurLame <= 0) {
            throw new Exception("La longueur de lame doit être positive.");
        }
        $this->_longueurLame = $longueurLame;
    }

    /**     get/set de tranchant     **/
    public function getTranchant(): int
    {
        return $this->_tranchant;
    }

    public function setTranchant(int $tranchant): void
    {
        if ($tranchant < 0) {
            throw new Exception("Le tranchant ne peut pas être négatif.");
        }
        $this->_tranchant = $tranchant;
    }

    #endregion

    #region Méthodes

    public function calculerPuissance(): float
    {
        return parent::calculerPuissance() + $this->_tranchant;
    }

    #endregion

    #region Méthodes redéfinies

    public function __toString(): string
    {
        return parent::__toString() . " | Épée (Lame: " . $this->_longueurLame . " cm, Tranchant:
" . $this->_tranchant . ")";
    }

    public function toString(): string
    {
        return $this->__toString();
    }

    #endregion
}
?>
```

1.5 Code et Captures d'Écran des Tests PHP

Script test.php mettant en œuvre la collection polymorphe et l'interception de l'exception :

Fichier : PHP/test.php

```php
<?php

require_once "Arme.class.php";
require_once "Epee.class.php";

$saut = (php_sapi_name() === 'cli') ? PHP_EOL : "<br>";

#region test 1 : Instanciation et Collection

// 1. Instanciation et Collection
$inventaire = [];

// Ajout d'une Arme : (1, "Gourdin", 15, 3.0)
$inventaire[] = new Arme(1, "Gourdin", 15, 3.0);

// Ajout de 2 Épées :
// (2, "Excalibur", 50, 2.5, 90.0, 25)
$inventaire[] = new Epee(2, "Excalibur", 50, 2.5, 90.0, 25);
// (3, "Katana", 40, 1.5, 75.0, 35)
$inventaire[] = new Epee(3, "Katana", 40, 1.5, 75.0, 35);

#endregion

#region test 2 : Boucle

// 2. Boucle
echo "=== 2. Boucle sur l'inventaire ===" . $saut . $saut;

foreach ($inventaire as $arme) {
    echo $arme . " -> Puissance calculée : " . $arme->calculerPuissance() . $saut;
}

echo $saut;

#endregion

#region Étape 3 : Test des Exceptions (Try / Catch)

// 3. Test des Exceptions (Try / Catch)
echo "=== 3. Test des Exceptions (Try / Catch) ===" . $saut;

try {
    // Tentative de création d'une épée invalide avec une lame négative : (4, "Épée brisée", 10,
1.0, -50.0, 5)
    $epeeInvalide = new Epee(4, "Épée brisée", 10, 1.0, -50.0, 5);
    echo "Succès : " . $epeeInvalide . $saut;
} catch (Exception $e) {
    echo "Exception capturée avec succès : " . $e->getMessage() . $saut;
}

#endregion
?>
```

Capture d'écran du résultat d'exécution du test :

```
==== 2. Boucle sur l'inventaire ====

[1] Gourdin - Dégâts: 15, Poids: 3 kg -> Puissance calculée : 5
[2] Excalibur - Dégâts: 50, Poids: 2.5 kg | Épée (Lame: 90 cm, Tranchant: 25) -> Puissance calculée : 45
[3] Katana - Dégâts: 40, Poids: 1.5 kg | Épée (Lame: 75 cm, Tranchant: 35) -> Puissance calculée : 61.666666666667

==== 3. Test des Exceptions (Try / Catch) ====
Exception capturée avec succès : La longueur de lame doit être positive.
```

Figure 4 : Exécution du test PHP (Collection polymorphe et exception capturée)

2) PARTIE C#

2.1 Présentation du Domaine Fonctionnel

Même domaine que la partie PHP : un inventaire de jeu vidéo composé d'armes et d'épées avec calcul dynamique de la puissance. Les classes C# reprennent exactement les mêmes noms, champs, validations et messages d'exception que les classes PHP.

• Classe mère (Arme) : id, nom, dégâts de base, poids. La méthode virtuelle CalculerPuissance() renvoie dégâts / poids.

• Classe fille (Epee) : hérite de Arme, ajoute la longueur de lame et le tranchant. CalculerPuissance() est redéfinie (override) : puissance de base + tranchant.

2.2 Diagramme de Classes (UML) C#

Diagramme de la bibliothèque de classes C# : relation d'héritage entre Epee (fille) et Arme (mère), avec propriétés, constructeurs et méthodes redéfinies.

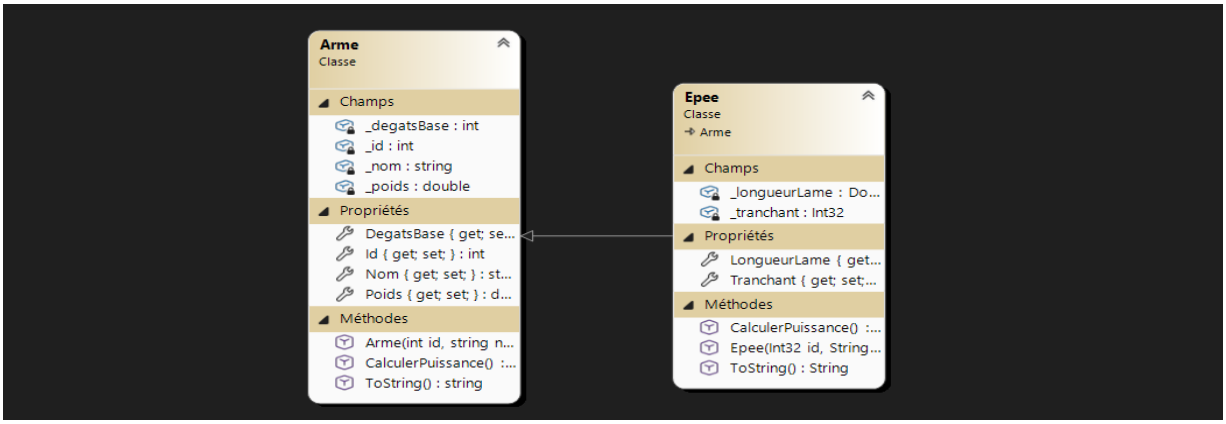


Figure 5 : Diagramme de classes UML C# (Arme et Epee)

2.3 Base de Données MySQL

Le C# réutilise la même base de données que le PHP (miniprojet_heritage) : les classes Arme et Epee sont mappées sur les tables arme (mère) et epee (fille). Aucune table supplémentaire n'a été créée. Script SQL et captures phpMyAdmin : voir 1.3 (Figures 2 et 3).

2.4 Code Source des Classes Métiers C#

Bibliothèque de classes (.dll, projet exos_poo_membres_heritage) développée sous Visual Studio : régions, propriétés, constructeurs, méthodes, exceptions et ToString() redéfini.

Fichier : exos_poo_membres_heritage/Arme.cs

```
using System;

namespace exos_poo_membres_heritage
{
    public class Arme
    {
        #region Champs

        private int _id;
        private string _nom;
        private int _degatsBase;
        private double _poids;

        #endregion

        #region Propriétés

        public int Id
        {
            get { return _id; }
            set { _id = value; }
        }

        public string Nom
        {
            get { return _nom; }
            set { _nom = value; }
        }

        public int DegatsBase
        {
            get { return _degatsBase; }
            set
            {
                if (value <= 0)
                    throw new ArgumentException("Les dégâts doivent être positifs.");
                _degatsBase = value;
            }
        }

        public double Poids
        {
            get { return _poids; }
            set
            {
                if (value <= 0)
                    throw new ArgumentException("Le poids doit être positif.");
                _poids = value;
            }
        }

        #endregion

        #region Constructeur

        public Arme(int id, string nom, int degatsBase, double poids)
        {
            Id = id;
```

```
            Nom = nom;
            DegatsBase = degatsBase;
            Poids = poids;
        }

        #endregion

        #region Méthodes

        public virtual double CalculerPuissance()
        {
            return _degatsBase / _poids;
        }

        #endregion

        #region Méthodes redéfinies

        public override string ToString()
        {
            return "[" + _id + "] " + _nom + " - Dégâts: " + _degatsBase + ", Poids: " + _poids +
" kg";
        }

        #endregion
    }
}
```

Fichier : exos_poo_membres_heritage/Epee.cs

```
using System;

namespace exos_poo_membres_heritage
{
    public class Epee : Arme
    {
        #region Champs

        private double _longueurLame;
        private int _tranchant;

        #endregion

        #region Propriétés

        public double LongueurLame
        {
            get { return _longueurLame; }
            set
            {
                if (value <= 0)
                    throw new ArgumentException("La longueur de lame doit être positive.");
                _longueurLame = value;
            }
        }

        public int Tranchant
        {
            get { return _tranchant; }
            set
            {
```

```
                if (value < 0)
                    throw new ArgumentException("Le tranchant ne peut pas être négatif.");
                _tranchant = value;
            }
        }

        #endregion

        #region Constructeur

        public Epee(int id, string nom, int degatsBase, double poids, double longueurLame, int
tranchant)
            : base(id, nom, degatsBase, poids)
        {
            LongueurLame = longueurLame;
            Tranchant = tranchant;
        }

        #endregion

        #region Méthodes

        public override double CalculerPuissance()
        {
            return base.CalculerPuissance() + _tranchant;
        }

        #endregion

        #region Méthodes redéfinies

        public override string ToString()
        {
            return base.ToString() + " | Épée (Lame: " + _longueurLame + " cm, Tranchant: " +
_tranchant + ")";
        }

        #endregion
    }
}
```

2.5 Code et Captures d'écran des Tests C#

Projet de test Windows Forms (exos_poo_membres_heritage_TESTS) : collection List<Arme> (polymorphisme), boucle sur l'inventaire et exception levée puis capturée avec try/catch. Les résultats s'affichent dans une zone de texte.

Fichier : exos_poo_membres_heritage_TESTS/Form1.cs

```
using System;
using System.Collections.Generic;
using System.Windows.Forms;
using exos_poo_membres_heritage;

namespace exos_poo_membres_heritage_TESTS
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
```

```
        private void Log(string s)
        {
            txtSortie.AppendText(s + Environment.NewLine);
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            #region test 1 : Instanciation et Collection

            var inventaire = new List<Arme>();
            inventaire.Add(new Arme(1, "Gourdin", 15, 3.0));
            inventaire.Add(new Epee(2, "Excalibur", 50, 2.5, 90.0, 25));
            inventaire.Add(new Epee(3, "Katana", 40, 1.5, 75.0, 35));

            #endregion

            #region test 2 : Boucle (polymorphisme)

            Log("=== 2. Boucle sur l'inventaire ===");
            foreach (Arme arme in inventaire)
                Log(arme + " -> Puissance calculée : " + arme.CalculerPuissance());

            #endregion

            #region test 3 : Exceptions (try / catch)

            Log("");
            Log("=== 3. Test des Exceptions (Try / Catch) ===");
            try
            {
                Epee epeeInvalide = new Epee(4, "Épée brisée", 10, 1.0, -50.0, 5);
                Log("Succès : " + epeeInvalide);
            }
            catch (Exception ex)
            {
                Log("Exception capturée avec succès : " + ex.Message);
            }

            #endregion
        }
    }
}
```

Capture d'écran du résultat d'exécution du test :

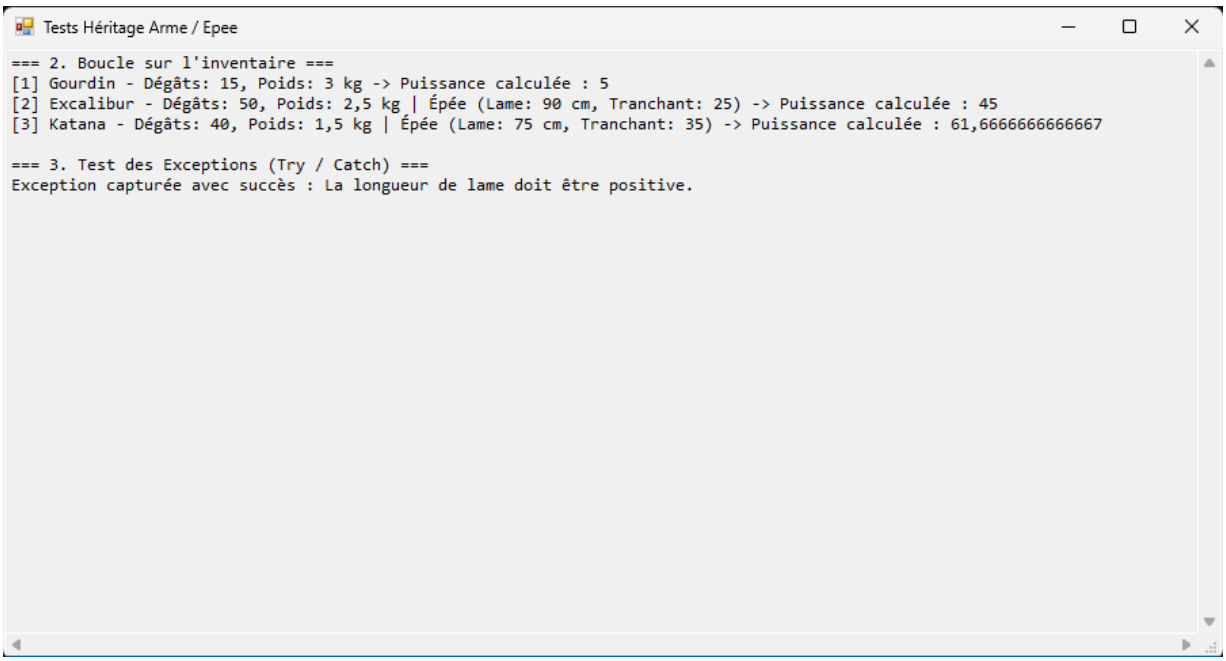


Figure 6 : Exécution du test C# (Collection polymorphe et exception capturée)